

Deep Research: Avatrada 57 Topic 053

Engineering Report: LLM Model Router

Component: LLM Model Router

Source: avatrada_57.pdf , Section 53

Date: October 26, 2023

Author: Autonomous Technical Researcher

Status: Analysis Complete

1.0 Executive Summary

This report provides a deep-dive analysis of the LLM Model Router system component. The primary function of this component is to intelligently and dynamically route user prompts to the most appropriate Large Language Model (LLM) based on a multi-faceted analysis of the prompt's content and the operational context. The specified routing logic aims to optimize for performance, cost, and security by directing prompts to Google's Gemini 3.0 Pro for complex tasks, Gemini 3.0 Flash for speed-critical tasks, and a local Gemma 2 instance for privacy-sensitive tasks. This analysis deconstructs the required architecture, proposes a concrete implementation strategy, and evaluates potential failure modes and optimizations.

2.0 Technical Deconstruction

The `ModelRouter` is a decision-making engine that sits between the user input and the LLM inference layer. Its architecture can be broken down into three core analysis modules and a final routing logic controller.

2.1 System Architecture

The logical flow of the system is as follows:

```
graph TD
  A[User Prompt + Context] --> B{ModelRouter};
  B --> C{1. Privacy Analysis};
  C -- Contains PII/Sensitive Data --> G[Route to Gemma 2 (Local)];
  C -- Clean --> D{2. Contextual Analysis};
  D -- Speed Critical (e.g., Market Open) --> H[Route to Gemini 3.0 Flash];
  D -- Not Speed Critical --> E{3. Complexity Analysis};
  E -- High Complexity --> F[Route to Gemini 3.0 Pro];
  E -- Low/Medium Complexity --> H;
```

2.2 Analysis Modules

1. Privacy Analysis Module: This is the first and most critical gate. Its sole purpose is to detect Personally Identifiable Information (PII) or sensitive proprietary data (e.g., trade logs, client IDs). * **Mechanisms:** * **Pattern Matching (Regex):** Fast detection of structured data like email addresses, phone numbers, credit card numbers, and specific trade log formats. * **Keyword Matching:** A predefined set of sensitive keywords ('trade_log', 'client_id', 'password', 'SSN'). * **Named Entity Recognition (NER):** A more advanced mechanism using a lightweight NLP model (e.g., a fine-tuned DistilBERT or spaCy's models) to identify entities like PERSON, ORG, and custom-defined entities like TRADE_ID .

2. Contextual Analysis Module: This module assesses external factors that are not present in the prompt text itself but are critical for routing decisions. * **Mechanism:** The router must accept a context object alongside the prompt string. This object contains metadata about the state of the application. * **Example Context Data:** * is_market_open: bool * user_role: str (e.g., 'trader', 'analyst', 'compliance') * request_source: str (e.g., 'batch_analysis_job', 'realtime_dashboard')

3. Complexity Analysis Module: This module evaluates the intrinsic complexity of the prompt to determine the required cognitive power of the LLM. * **Mechanisms:** * **Quantitative Heuristics:** * **Token Count:** Longer prompts often correlate with higher complexity. A simple threshold (e.g., > 500 tokens) can be a first-pass filter. * **Syntactic Complexity:** Measures like sentence length or the number of subordinate clauses could be used, though this adds processing

overhead. * **Qualitative Heuristics (Keyword-based):** * **High Complexity Keywords:** 'analyze', 'deep dive', 'strategize', 'forecast', 'model', 'explain in detail', 'write code for'. * **Low Complexity Keywords:** 'summarize', 'quick sentiment', 'classify', 'extract', 'what is'.

2.3 Routing Logic

The decision-making process follows a strict priority order: **Privacy > Context (Speed) > Complexity**. This ensures that security is never compromised and time-sensitive tasks are always prioritized.

```
// Pseudocode for the core routing decision
function decide_route(prompt, context):
    if privacy_module.contains_pii(prompt):
        return "Gemma 2"

    if context_module.is_speed_critical(context):
        return "Gemini 3.0 Flash"

    if complexity_module.is_high_complexity(prompt):
        return "Gemini 3.0 Pro"
    else:
        // Default to the faster/cheaper model for simple tasks
        return "Gemini 3.0 Flash"
```

3.0 Implementation Strategy

This section outlines a practical approach to building the `ModelRouter` class in Python.

3.1 Libraries and Dependencies

- **LLM Clients:** `google-generativeai` for Gemini models. A local HTTP client or a library like `transformers` for Gemma 2.
- **PII Detection:** `presidio-analyzer` for robust PII detection, or `spacy` for general NER. `re` for basic regex.

- **Tokenization:** `tiktoken` or a model-specific tokenizer to count tokens accurately.
- **Configuration:** A `config.yaml` or `.env` file to manage API keys, model names, and thresholds.

3.2 Class Design and API Switching

A class-based approach encapsulates the logic cleanly. API switching can be handled by abstracting the model clients behind a unified interface.

```
import re
import os
import google.generativeai as genai
from presidio_analyzer import AnalyzerEngine

# --- Configuration (ideally from a config file) ---
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GEMMA_LOCAL_ENDPOINT = "http://localhost:8080/v1/chat/completions"

COMPLEXITY_TOKEN_THRESHOLD = 500
HIGH_COMPLEXITY_KEYWORDS = {'analyze', 'deep dive', 'strategize', 'forecast',
                             'model'}
SENSITIVE_KEYWORDS = {'trade log', 'client id', 'pii'}

# --- Model Client Setup ---
genai.configure(api_key=GEMINI_API_KEY)

class ModelRouter:
    def __init__(self):
        # 1. Initialize analysis tools
        self.pii_analyzer = AnalyzerEngine()

        # 2. Initialize model clients/definitions
        self.models = {
            "gemini-3.0-pro": genai.GenerativeModel('gemini-pro'),
            "gemini-3.0-flash": genai.GenerativeModel('gemini-flash'),
            "gemma-2-local": self._get_local_gemma_client()
        }
        print("ModelRouter initialized.")
```

```

def _get_local_gemma_client(self):
    # Placeholder for a client to interact with a local Gemma instance
    # e.g., using the 'requests' library
    pass

def _contains_pii(self, prompt: str) -> bool:
    """Checks for PII using Presidio and keyword matching."""
    if any(keyword in prompt.lower() for keyword in SENSITIVE_KEYWORDS):
        return True

    analyzer_results = self.pii_analyzer.analyze(text=prompt, language='en')
    return len(analyzer_results) > 0

def _is_high_complexity(self, prompt: str) -> bool:
    """Analyzes prompt complexity based on keywords and token length."""
    # Simple token count (can be replaced with a proper tokenizer)
    token_count = len(prompt.split())
    if token_count > COMPLEXITY_TOKEN_THRESHOLD:
        return True

    if any(keyword in prompt.lower() for keyword in
HIGH_COMPLEXITY_KEYWORDS):
        return True

    return False

def route(self, prompt: str, context: dict = None) -> str:
    """
    Determines the appropriate model based on a prioritized set of rules.
    Priority: Privacy > Context (Speed) > Complexity
    """
    if context is None:
        context = {}

    # 1. Privacy Gate
    if self._contains_pii(prompt):
        print("Routing decision: PII detected -> Gemma 2 (Local)")
        return "gemma-2-local"

```

```

        # 2. Context Gate (Speed)
        if context.get('is_market_open', False):
            print("Routing decision: Speed critical (Market Open) -> Gemini 3.0 Flash")
            return "gemini-3.0-flash"

        # 3. Complexity Gate
        if self._is_high_complexity(prompt):
            print("Routing decision: High complexity -> Gemini 3.0 Pro")
            return "gemini-3.0-pro"

        # 4. Default Route
        print("Routing decision: Default (Low/Medium Complexity) -> Gemini 3.0 Flash")
        return "gemini-3.0-flash"

    def execute_prompt(self, prompt: str, context: dict = None):
        """Routes and then executes the prompt against the chosen model."""
        model_name = self.route(prompt, context)

        # API Switching Logic
        if model_name.startswith("gemini"):
            client = self.models[model_name]
            response = client.generate_content(prompt)
            return response.text
        elif model_name == "gemma-2-local":
            # Logic to call the local Gemma endpoint
            # response = requests.post(GEMMA_LOCAL_ENDPOINT, json={"prompt":
prompt})
            # return response.json()[0]['choices'][0]['message']['content']
            return "Response from local Gemma 2." # Placeholder
        else:
            raise ValueError(f"Unknown model name: {model_name}")

# --- Example Usage ---
# router = ModelRouter()
# complex_prompt = "Deep dive into the Q3 financial statements and strategize a new investment model."
# router.execute_prompt(complex_prompt)

```

```
# fast_prompt = "Quick sentiment on AAPL stock now."
# market_context = {"is_market_open": True}
# router.execute_prompt(fast_prompt, context=market_context)

# private_prompt = "Fetch the trade log for client_id 8A7-B2-C3."
# router.execute_prompt(private_prompt)
```

4.0 Critical Analysis

While the proposed system is robust, it is essential to consider its limitations, potential failure modes, and avenues for optimization.

4.1 Potential Failure Modes & Edge Cases

1. PII Detection Failure:

- **False Negatives:** The most critical failure. If the PII scanner misses sensitive data (e.g., an unusual ID format or implied PII), a private prompt could be sent to a public cloud model, causing a data breach. The reliability of the PII detection module is the system's most significant vulnerability.
- **False Positives:** A non-sensitive prompt is incorrectly flagged as PII and sent to the weaker local model, resulting in a poor-quality response and user frustration.

2. Incorrect Complexity Classification:

- A short but conceptually difficult prompt (e.g., "Explain quantum entanglement in one sentence") might be misclassified as "low complexity" and routed to Flash, yielding a superficial answer.
- Conversely, a long but simple prompt (e.g., a large block of text for summarization) might be routed to the expensive Pro model unnecessarily.

3. Latency Overhead:

- The routing logic itself introduces latency. A complex PII scan could take several hundred milliseconds, potentially negating the speed advantage of routing to Gemini Flash for a "fast" query. The performance of each analysis module must be benchmarked.

4. Static Heuristics:

- The keyword lists and token thresholds are static and brittle. They require manual tuning and can be easily "gamed" or may not generalize well to new types of prompts.

4.2 Optimizations and Enhancements

1. ML-based Router Classifier:

- Replace the static heuristic rules with a small, fine-tuned classification model (e.g., a distilled-BERT). This model would be trained on a dataset of prompts labeled with the optimal destination model (`Pro` , `Flash` , `Gemma`). This approach can learn the nuances of complexity and intent far better than static rules. The training feedback loop could even be automated based on user satisfaction scores or response quality metrics.

2. Semantic Caching:

- Before routing, convert the incoming prompt into a vector embedding. Check a vector database for semantically similar prompts that have been answered recently. If a close match is found, return the cached response immediately, bypassing the LLM call entirely. This dramatically reduces both latency and cost.

3. Cost-Based Routing:

- Enhance the `context` object to include a `max_cost` or `priority_level` parameter. The router could then factor in the cost-per-token of each model into its decision, choosing the most powerful model that fits within the user's budget for that query.

4. Dynamic Fallback and Health Checks:

- The router should incorporate health checks for all model endpoints. If the primary chosen model (e.g., Gemini Pro) is down or responding with errors, the router should automatically fall back to a secondary option (e.g., Gemini Flash) and log the event.

5. Hybrid PII Analysis:

- For maximum security, use a tiered PII detection approach. First, run a fast regex/keyword scan. If it passes, but the prompt is still flagged for a potential cloud model, run a more thorough (and slower) NER-based analysis as a second check. This balances speed and security.